Improving HTTP/2 Race-Condition Synchronization Through TCP Segment Reordering

Alexandru-Florin Călin

August 17, 2026

Abstract

Single-packet synchronization techniques for HTTP/2 race-condition testing have been shown to reduce sensitivity to network jitter [6]. However, their effectiveness remains influenced by transport-layer fragmentation behavior in real-world TCP/IP stacks.

Building upon Ryotak's First Sequence Sync technique [7], we extend the underlying transport-layer synchronization concept to TLS-enabled HTTP/2 environments. We present a mechanism that selectively delays and reorders TCP segments during transmission, providing finer control over the arrival timing of fragmented HTTP/2 request payloads without requiring modifications to the application-layer protocol.

We implement and evaluate our approach in a distributed cloud testbed using geographically separated attacker and server instances. Across 50 experimental runs involving 5,000 concurrent requests per run, our method reduced the interval between the first and last request processed by the server by approximately 4.64× relative to a standard last-byte synchronization baseline. The mean synchronization window decreased from 263.8 ms to 56.8 ms, corresponding to a 78.5% reduction.

Our results suggest that transport-layer packet ordering can significantly influence synchronization precision in HTTP/2 race-condition experiments and may improve the reproducibility of timing-sensitive security evaluations.

1 Introduction

Race-condition vulnerabilities arise when the correctness or security of a system depends on the relative timing of concurrent operations. In web applications and distributed systems, successful reproduction of such vulnerabilities often requires multiple requests to be processed within a very narrow time interval, often hard to reproduce due to variability in network conditions. As a result, the reliability of race-condition testing is strongly influenced by the ability to synchronize request delivery.

Recent work has introduced synchronization techniques that improve the precision of concurrent request delivery over HTTP/2. In particular, the single-packet technique introduced by James Kettle (2023) [6] reduces sensitivity to network jitter by leveraging HTTP/2 request construction such that the final bytes of multiple requests are transmitted together within a single TCP segment, improving the alignment of request completion times.

However, the effectiveness of this approach is influenced by link-layer constraints, namely Ethernet Maximum Transmission Unit (MTU) limits (typically 1500 bytes), which restrict the amount of HTTP/2 data that can be transmitted within a single Ethernet frame. As a result, a single frame may only carry enough data to complete approximately 20–30 HTTP requests under typical configurations.

In this paper, we investigate whether controlled TCP segment reordering can be used to improve synchronization precision in HTTP/2 request streams. Rather than relying solely on application-layer synchronization, our approach manipulates the delivery order of selected TCP segments during transmission. By delaying and subsequently releasing specific packets, the technique seeks to reduce the spread between the earliest and latest requests processed by the target server.

To evaluate the proposed method, we implemented a prototype consisting of a TLS-enabled HTTP/2 client and server. The system was deployed across two geographically separated Google Cloud Platform instances, with the client (attacker) hosted on an n2-standard-2 instance and the target server hosted on a c4-standard-24 instance in a different region. This setup introduces realistic inter-region latency and network variability.

2 Design

The proposed technique builds upon the First Sequence Sync synchronization method described by Ryotak [7]. First Sequence Sync exploits TCP's in-order delivery semantics by intentionally delaying the transmission of the segment containing the earliest TCP sequence number. Because TCP does not deliver later segments to the application until missing data has been received, delaying the first segment causes subsequently transmitted segments to be buffered by the receiver until the missing segment arrives.

This property can be used to synchronize the completion of multiple TCP segments at the same time. Once all remaining segments have reached the target, the delayed segment is released, causing the buffered data to become available to the application within a narrow

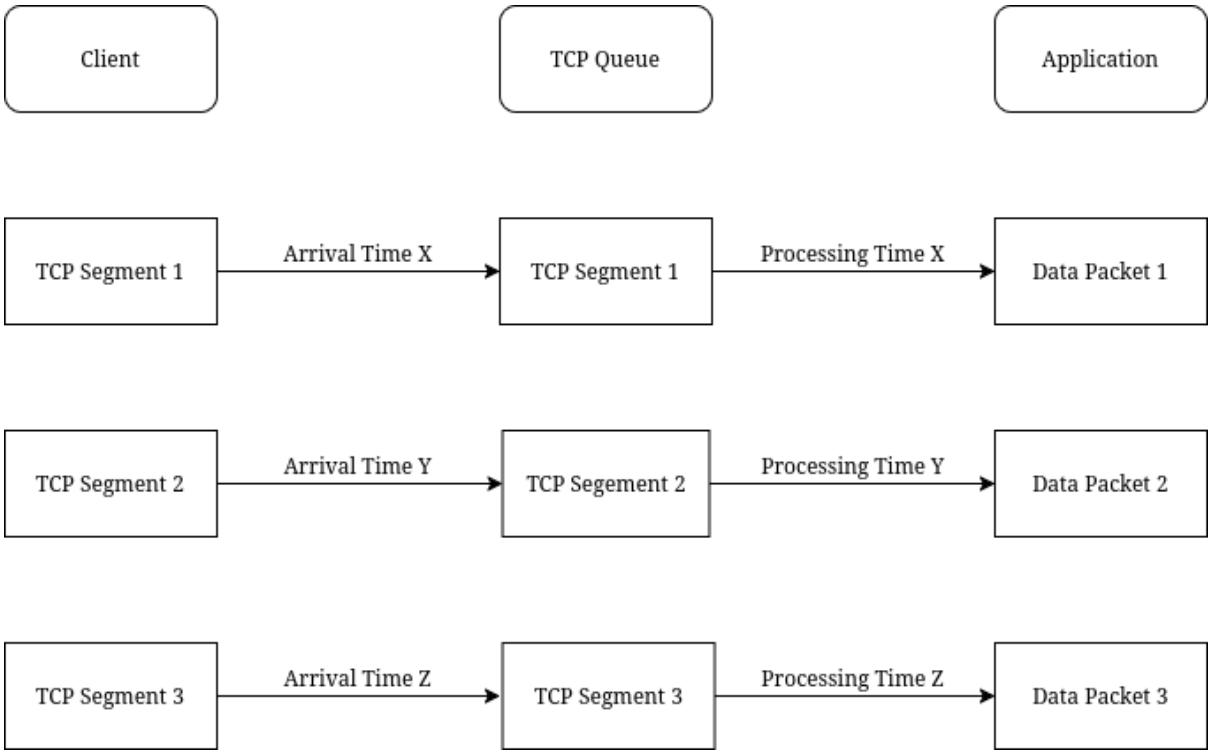

Figure 1: Regular TCP segment traffic.

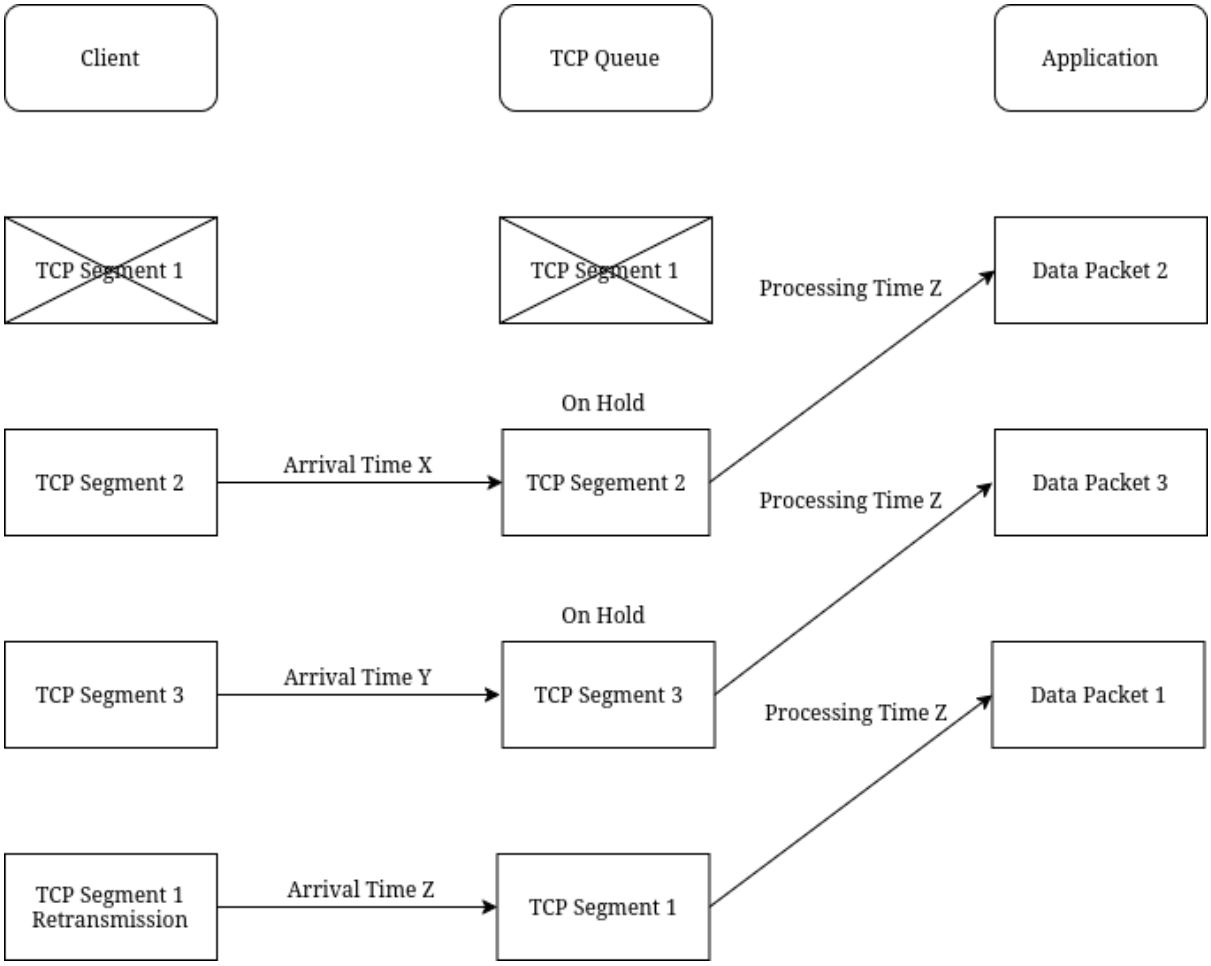

Figure 2: TCP segment reordering.

time interval.

Our work extends this concept to a TLS-enabled HTTP/2 environment and evaluates its effectiveness at a larger scale.

We leverage the h2spacex framework [1] as a baseline implementation of TLS-enabled HTTP/2 with last-byte synchronization in Python. Building on this system, we introduce a transport-layer extension that reorders TCP segments to further shape end-to-end delivery timing under TCP's in-order delivery guarantees.

2.1 System Overview

The system consists of three components: a TLS-enabled HTTP/2 client, a packet interception layer, and a remote HTTP/2 server. The client generates batches of concurrent HTTP/2 requests, which are segmented at the TCP layer during transmission. The interception layer operates on outbound TCP segments, allowing controlled manipulation of packet scheduling before delivery to the network. The server processes incoming HTTP/2 requests and records per-request arrival timestamps for later analysis.

2.2 Controlled TCP Segment Reordering

The key idea of our approach is to decouple the transmission order of TCP segments from their application-layer generation order. In particular, we identify the segment corresponding to the initial portion of the last-byte HTTP/2 data and temporarily delay its transmission while allowing subsequent segments to be sent immediately.

This results in a controlled out-of-order delivery pattern at the TCP layer, where later-generated segments arrive at the receiver earlier than the initial segment. Because TCP enforces in-order delivery to the application layer, these later segments are buffered until the missing segment is received. Once the delayed segment is released, the buffered segments are delivered in a narrow time window, effectively compressing the request completion interval Figure 2.

2.3 Execution Flow

For each experiment run, the system performs the following steps:

1. The client generates N concurrent HTTP/2 requests.
2. The last-byte portions of all requests are isolated at the application layer level for synchronization control.
3. The bulk of the HTTP/2 requests are transmitted to the server.
4. The interception layer selectively delays transmission of the first TCP segment containing last-byte data.
5. The rest of the TCP segments containing the last bytes of the HTTP/2 requests are sent.
6. After a set amount of time, the delayed segment is released.
7. The server receives buffered TCP segments, which are delivered to the application layer in a compressed time window.

2.4 Implementation Considerations

The proposed system is implemented using a TLS-enabled HTTP/2 client built on top of the h2spacex [1] framework, which provides low-level control over HTTP/2 frame generation and transmission. This allows explicit control over request segmentation and the construction of concurrent request streams.

Packet interception is implemented at the network layer using a Netfilter-based queueing mechanism [2], which enables inspection and controlled modification of outbound TCP segments.

The system is designed to operate in real-world network conditions, including inter-region latency and variable packet scheduling introduced by cloud infrastructure.

3 Implementation

This section describes the implementation of the proposed system, which consists of three main components: a TLS-enabled HTTP/2 client, a TCP packet interception layer, and a server-side measurement framework. The system is implemented in Python and Go and deployed across two geographically separated Google Cloud instances.

3.1 Client-side HTTP/2 TLS Implementation

The benchmarking environment is based on Debian GNU/Linux 12 (bookworm).

The mechanism assumes visibility and control over outbound TCP segments at the sender side and therefore relies on kernel-level packet interception capabilities. All outgoing TCP segments destined for the target server are redirected to a NetfilterQueue instance using an iptables OUTPUT rule targeting TCP traffic on port 443 (e.g., `iptables -I OUTPUT -p tcp -d 34.176.133.79 -dport 443 -j NFQUEUE -queue-num 0`).

The client launches a dedicated thread to handle NetfilterQueue processing. Incoming packets are processed through a `process_packet(pkt)` callback function, which inspects and selectively manipulates TCP segments as part of the synchronization mechanism.

The `h2spacex` framework initializes N HTTP/2 streams using a generated stream identifier list (i.e., `generate_stream_ids(number_of_streams = N)`), and constructs N HTTP requests, each split into a bulk data fragment and a final byte fragment.

The framework first transmits the bulk portions of all requests, which are sent to the target server in rapid succession. It then pauses transmission for a short interval of X seconds, allowing in-flight packets to propagate and reach the destination before the last-byte fragments are released.

When transmission of the final-byte fragments begins, the `process_packet` function records the sequence number of the first outgoing TCP segment and selectively drops that segment, including any retransmissions associated with it. This behavior is maintained for a fixed duration of Y seconds to allow the rest of the outgoing TCP segments to reach their destination.

After this interval, packet filtering is disabled at the interception layer, allowing the `h2spacex` client to retransmit the previously withheld TCP segment. Once this segment is delivered, the TCP sequence is completed, enabling the buffered HTTP/2 fragments to be processed by the server within a narrow time window.

No artificial delays are introduced in the request generation process beyond the controlled pauses used for synchronization. Packet ordering and delivery timing are therefore governed by the underlying TCP/IP stack and cloud network conditions.

3.2 Server-side Implementation

The server is implemented in Go using the `golang.org/x/net/http2` [8] library with TLS enabled. HTTPS over HTTP/2 is configured using a locally generated self-signed certificate, loaded at runtime via `tls.LoadX509KeyPair`. This avoids reliance on external certificate authorities while preserving encrypted transport.

The server exposes a single endpoint at `/api/data`, which accepts HTTP POST requests. Incoming request bodies are parsed as URL-encoded form data using `url.ParseQuery`, but are not otherwise processed.

Per-request arrival timing is recorded using `time.Now().UnixMilli()`. The timestamp of the first received request is stored as a baseline, while each subsequent request updates the most recent arrival timestamp. A global counter tracks the number of requests received per run.

The synchronization window is defined as the difference between the first and last recorded request timestamps within a run. This value is computed upon termination and logged together with the total request count.

The HTTP/2 server is configured with a high `MaxConcurrentStreams` limit to prevent server-side stream throttling during high-concurrency experiments.

4 Evaluation

4.1 Experimental Setup

Experiments are conducted using two geographically distributed Google Cloud Platform (GCP) instances located in separate regions: Sydney and Santiago. The client system is deployed on an `n2-standard-2` instance (2 vCPUs, 8 GB memory) in Santiago, while the target server is deployed on a `c4-standard-24` instance (24 vCPUs, 90 GB memory) in Sydney.

This configuration introduces realistic inter-region network latency and variability, reflecting conditions commonly observed in wide-area network deployments. The client operates as the attacker-side machine responsible for generating and transmitting HTTP/2 request batches, while the server acts as the victim-side endpoint responsible for processing incoming requests and recording timing measurements.

4.2 Metrics

Synchronization Window. The primary metric used to evaluate synchronization precision is the *synchronization window*, defined as the time interval between the earliest and latest observed request arrivals within a single experiment run. Formally, let t_i denote the arrival timestamp (in milliseconds) of request i. The synchronization window W is defined as:

$$W = \max_{i \in [1,N]} (t_i) - \min_{i \in [1,N]} (t_i)$$

where N is the number of requests issued per run.

A smaller synchronization window indicates tighter clustering of request arrivals at the server, corresponding to higher synchronization precision.

Successful Requests

We define a successful request as any HTTP/2 request that receives a valid response from the server. A response is considered valid if the server returns an HTTP status code and completes the request without connection termination or read timeout.

This metric is used to ensure that improvements in synchronization precision do not come at the cost of reduced reliability in request delivery. In particular, we verify that variations in packet scheduling and TCP segment reordering do not significantly impact the ability of the server to process all transmitted requests.

Across all experiments, successful requests are determined based on server-side logging of completed HTTP/2 handlers, ensuring that only fully processed requests are included in the final measurement set.

Measurement Methodology. Arrival timestamps are recorded at the application layer on the server using `time.Now().UnixMilli()` upon receipt of each HTTP/2 request. For each experiment run, the synchronization window is computed post hoc from the collected timestamp set.

4.3 Baseline

The proposed technique is evaluated against a standard HTTP/2 last-byte synchronization baseline. In the baseline configuration, requests are generated using the `h2spacex` framework and split into bulk data fragments and final-byte fragments. After transmission of the bulk data, the final-byte fragments are released simultaneously, causing all requests to become complete at approximately the same time.

Unlike the proposed approach, the baseline does not perform any transport-layer packet manipulation. TCP segments are transmitted according to the operating system's default networking behavior, and no packet interception, delay, or reordering is applied.

To ensure a fair comparison, both the baseline and the proposed technique use identical experimental conditions, including the same client and server implementations, cloud infrastructure, HTTP/2 configuration, TLS settings, request workload, and number of concurrent requests. The only difference between the two configurations is the introduction of controlled TCP segment reordering in the proposed method.

Synchronization precision is evaluated using the synchronization window metric defined in Section 4.2. Improvements are reported as reductions in the measured synchronization window relative to the baseline configuration.

4.4 Results

We evaluated the proposed TCP segment reordering technique across 50 independent experimental runs against 50 experiment runs with the baseline technique. Each run consisted of 5,000 concurrent HTTP/2 requests transmitted over a single TLS-encrypted connection between the Santiago client instance and the Sydney server instance.

For each run, we recorded the synchronization window and the total number of successful requests. The resulting values were compared against those obtained using the baseline last-byte synchronization technique under identical experimental conditions.

Across all experiments, the proposed approach consistently produced smaller synchronization windows than the baseline configuration. On average, the synchronization window was reduced by a factor of 4.64×, indicating substantially tighter clustering of request arrivals at the target server. Figure 3, 4

Table 1: Summary of Synchronization Window and Successful Requests

Metric	Mean	Median
Sync Window (Reordered TCP)	56.8	56.0
Sync Window	263.76	261.0
Requests (Reordered TCP)	4989.08	4989.0
Requests	4989.68	4991.0

Table 2: Standard Deviation

Metric	Standard Deviation
Sync Window (Reordered TCP)	2.529
Sync Window	10.264
Requests (Reordered TCP)	3.018
Requests	3.608

These results suggest that controlled TCP segment reordering can significantly improve synchronization precision beyond what is achievable through application-layer last-byte synchronization alone. By exploiting TCP's in-order delivery semantics, the proposed method causes a larger fraction of requests to become available to the application layer within a narrower time interval without a significant effect on request completion Figure 5, 6.

Table 3: 95% Confidence Intervals

Metric	95% Confidence
Sync Window (Reordered TCP)	[56.02, 57.48]
Sync Window (Benchmark)	[260.80, 266.70]
Requests (Reordered TCP)	[4988.25, 4989.98]
Requests	[4988.61, 4990.68]

Table 4: Max and Min Values

Metric	Max	Min
Sync Window (Reordered TCP)	63	52
Sync Window (Benchmark)	300	257
Requests (Reordered TCP)	4994	4983
Requests (Benchmark)	4996	4980

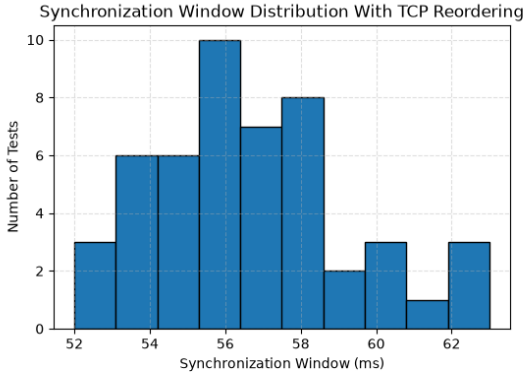

Figure 3: Synchronization Window distribution with TCP segment reordering.

The confidence intervals of the two synchronization-window distributions do not overlap, indicating a statistically significant difference between the baseline and reordered configurations Figure 5, 6.

The observed improvement was consistent across repeated runs despite the presence of inter-region network latency and variability introduced by the underlying cloud infrastructure. This indicates that the synchronization gains are not solely the result of favorable network conditions, but arise from the transport-layer behavior induced by the reordering mechanism.

Overall, the results demonstrate that transport-layer control can serve as an effective complement to existing HTTP/2 synchronization techniques, enabling tighter request alignment at scales substantially exceeding the limits imposed by single-packet synchronization methods.

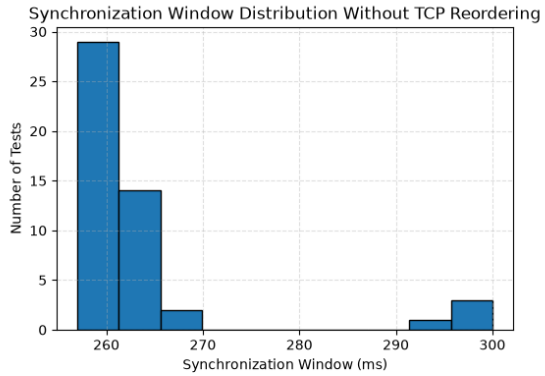

Figure 4: Synchronization Window distribution without TCP segment reordering.

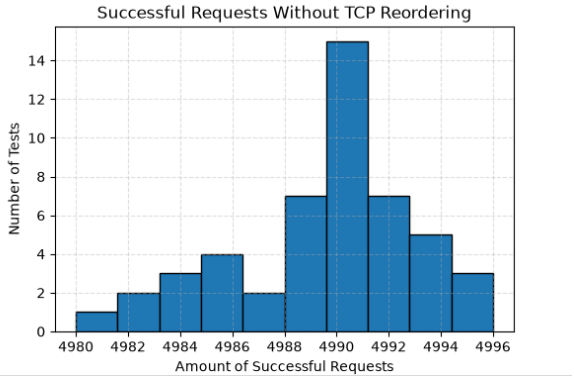

Figure 5: Successful Requests Without TCP Reordering.

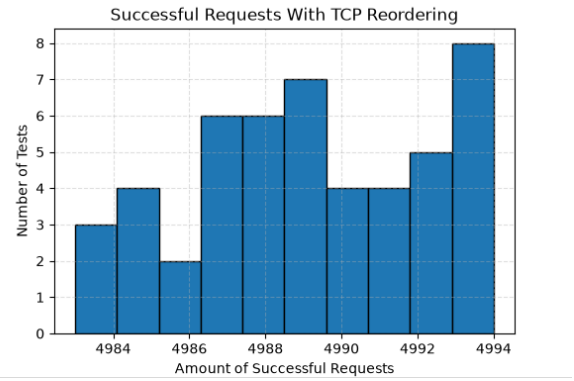

Figure 6: Successful Requests With TCP Reordering.

5 Limitations

The techniques and implementation described in this paper have several limitations.

First, the effectiveness of the proposed technique is influenced by the maximum number of concurrent HTTP/2 streams supported by the target implementation. While the experimental server was configured with a very high `MaxConcurrentStreams` value, many production HTTP/2 deployments enforce substantially lower limits. Consequently, the scale at which the technique can be applied may vary depending on the target server and HTTP/2 implementation.

Second, the packet interception logic operates purely at the TCP layer and does not decode HTTP/2 frames or TLS records. Instead, the implementation identifies the sequence-critical segment based on transmission order and TCP sequence numbers. While this approach proved sufficient for the evaluated workloads, a protocol-aware implementation could directly identify segments containing final-byte HTTP/2 fragments and provide more robust synchronization control.

Third, the current prototype relies on fixed timing delays (`time.sleep()`) to allow packets to propagate through the network before releasing the withheld TCP segment. This approach introduces implementation-specific tuning parameters and may not be optimal under varying network conditions. A more robust design could dynamically determine when all relevant segments have reached the receiver or use feedback-driven synchronization mechanisms.

6 Discussion

The observed results were obtained on Google Cloud infrastructure, which employs highly optimized networking and routing mechanisms between geographically distributed server instances. As a result, the measured network latency between the attacker-client instance and the victim-server instance may be lower and more stable than in typical Internet deployments. This may influence the observed synchronization windows and, consequently, the measured effectiveness of the proposed technique. Therefore, the results should be interpreted within the context of a highly optimized cloud environment. In more heterogeneous, congested, or high-latency networks, performance characteristics may differ. Future work should incorporate artificial latency injection.

The proposed technique improves the synchronization of a large number of concurrent requests, potentially increasing the effectiveness of brute-force attacks against web applications where the rate limiting algorithm is affected by race condition vulnerabilities. In particular, it can facilitate attacks against mechanisms such as one-time passwords, six-digit PIN verification systems, and rate-limited authentication workflows when their security relies on the assumption that concurrent requests are processed sequentially by the rate-limiting algorithm.

While the present evaluation focuses on synchronization precision and request processing windows, future work could investigate the effectiveness of the proposed technique in realistic race-condition scenarios. One potential direction is the evaluation of applications that rely on one-time verification codes or similar stateful authentication mechanisms, where multiple concurrent requests may compete to consume a shared resource or validation token.

Such experiments would provide insight into the practical impact of improved synchronization on real-world race-condition vulnerabilities and would allow comparison against existing synchronization techniques under application-level conditions. Conducting these evaluations in a controlled laboratory environment could help quantify the relationship between synchronization window reduction and race-condition exploitability while avoiding reliance on synthetic timing metrics alone.

In addition to synchronization window measurements, we evaluated the impact of the proposed technique on request completion rates. Across all experimental runs, the number of successful requests remained comparable between the baseline last-byte synchronization method and the TCP segment reordering approach. The observed differences in success rate were small relative to the total number of requests issued and did not exhibit a consistent trend across runs. These results suggest that the reduction in synchronization window is not achieved at the expense of request delivery reliability, and that the proposed packet reordering mechanism has a negligible effect on the overall proportion of successfully processed requests.

7 Conclusion

This paper investigated whether controlled TCP segment reordering can improve synchronization precision in HTTP/2 race-condition testing. We presented a transport-layer mechanism that selectively delays and releases TCP segments to influence the timing of HTTP/2 request completion without modifying the application-layer protocol.

We implemented a prototype system using a TLS-enabled HTTP/2 client and server deployed across geographically distributed Google Cloud instances. Across 50 experimental runs with 5,000 concurrent requests per run, the proposed method reduced the synchronization window by a factor of $4.64\times$ compared to a standard last-byte synchronization baseline.

The results demonstrate that transport-layer behavior can significantly reduce the synchronization window across high-latency, geographically distributed environments, improving the precision of concurrent request delivery in race-condition evaluation settings. The proposed method achieved this improvement without materially affecting request completion rates.

References

[1] h2spacex: Http/2 tls research framework. `https://github.com/nxenon/h2spacex`.

[2] Netfilterqueue python binding. `https://pypi.org/project/NetfilterQueue/`.

[3] Scapy: Packet manipulation library. `https://scapy.net`.

[4] IETF. Rfc 9113: Http/2. `https://www.rfc-editor.org/rfc/rfc9113`, 2022.

[5] IETF. Rfc 9293: Transmission control protocol (tcp). `https://www.rfc-editor.org/rfc/rfc9293`, 2022.

[6] James Kettle. Smashing the state machine: the true potential of web race conditions. `https://portswigger.net/research/smashing-the-state-machine`, 2023. PortSwigger Research.

[7] RyotaK. Beyond the limit: Expanding single-packet race condition, 2024.

[8] The Go Authors. golang.org/x/net/http2. `https://pkg.go.dev/golang.org/x/net/http2`, 2025. Accessed: 2026-06-16.